RESMİ MİMARİ VE KOD HARİTASI REFERANSI

AGENT OBSERVATORY

Tüm Dosyaların Fonksiyonel ve Teknik Görev Rehberi

Claude Code, OpenAI Codex, OpenCode DeepSeek, Google Antigravity ve yerel CLI yapay zekâ ajanları için multi-agent telemetri, kanca izleme, TUI ve web operasyon orkestratörü.

Proje Adı:	**Agent Observatory**
Sürüm:	v1.0.0-rc.1 (Production Hardened)
Doküman Türü:	%100 Kapsamlı Dosya Açıklama Kataloğu
Hedef Platform:	Cross-Platform (Windows 11 / macOS / Linux)
Tarih:	29 Ekim 2026 (Son Güncelleme)
Durum:	Tüm testler (%100) doğrulanmış ve üretime hazır

1. Sisteme Genel Bakış ve Mimari Katmanlar

Agent Observatory, geliştiricinin bilgisayarında çalışan farklı yapay zekâ kodlama motorlarını (Claude Code, OpenAI Codex, OpenCode, Google Antigravity, Command Code, Freebuff) tek bir çatı altında birleştiren, izleyen, yönlendiren ve güvenliğini sağlayan yerel-öncelikli (Local-First) bir gözlemlenebilirlik platformudur.

> **Üç Seviyeli Telemetri ve Gözlem Modeli (3-Tier Model)**
>
> - **Tier 1 (Doğrudan Yerel Akış / Native Stream):** Observatory'nin kendi CLI veya API motoru üzerinden `child_process.spawn` ile başlattığı ajanların standart çıktı (stdout/stderr) akışını anlık NDJSON parçaları olarak yakalaması.
> - **Tier 2 (Kanca ve OTel / Hooks):** Ajanların yaşam döngüsü kancalarından (`PreToolUse`, `PostToolUse`, `SubagentStart`, `Stop`) gelen yapılandırılmış olayların yakalanması.
> - **Tier 3 (Arka Plan Dosya İzleme / Ingest Watcher):** Geliştiricinin terminalde elle bağımsız başlattığı veya Antigravity IDE içinden tetiklediği harici oturumların log dosyalarını (`transcript.jsonl`, `opencode.db`, `projects/**/*.jsonl`, `sessions/**/*.jsonl`) salt-okunur olarak dinleyip merkezi SQLite veritabanına akıtması.

Çekirdek Dizin Dağılımı

Proje modüler, yüksek uyumlu ve tek sorumluluk prensibine (SRP) tam sadık bir dizin mimarisine sahiptir:

- `src/core/`: Olay modelleri, ACID SQLite depolama, deterministik durum makineleri ve DAG çizelgeleyici.
- `src/adapters/`: Farklı CLI ajanlarının komut satırı argümanlarını ve çıktı formatlarını soyutlayan adaptörler.
- `src/runtime/`: Süreç yönetimi, doğrudan çalıştırma, paralel filo orkestrasyonu, Git worktree izolasyonu ve kaynak kilitleri.
- `src/ingest/`: Harici terminal oturumlarını otomatik yakalayan arka plan dinleyicileri.
- `src/commands/`: HTTP/SSE sunucusu, web API'leri, sistem doktoru ve uzaktan eşleştirme komutları.
- `src/ui/`: Terminal Ağacı (TUI Tree), ASCII ofis görünümü ve terminal transcript formatlayıcıları.
- `src/mcp/`: Resmi Model Context Protocol sunucu modülü (Antigravity entegrasyonu).
- `web/`: Tam ekran piksel sanatlı ofis simülatörü, 4 sekmeli HUD (Kanca akışı, konsol, Mindwalk, görevler), DAG ve Fleet panoları.
- `nexus_ai_office/`: Deneysel 3D ortam ve Python Curses tabanlı ofis simülasyonu.

2. Kök Dizin ve Standart Yönetim Dosyaları

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`package.json`	PAKET YAPILANDIRMASI	Proje bağımlılıklarını (Node.js v22+ yerel SQLite modülü `node:sqlite`, `@modelcontextprotocol/sdk`), TypeScript derleme betiklerini, test komutlarını ve CLI binary eşlemelerini (`observatory`) tanımlar. Harici ağır bağımlılık barındırmaz.
`tsconfig.json`	TYPESCRİPT DERLEYİCİ	Node.js `--experimental-strip-types` ile uyumlu, modern ESNext modül çözünürlüğü ve katı tip denetim kurallarını belirler.
`.gravityguard.json`	YÖNETİŞİM & DENETİM	GravityGuard mimari ve kalite kurallarını barındırır: 200 satır çekirdek dosya sınırları, sessiz hata yutma yasakları (G1) ve körü körüne dosya üzerine yazma engelleri (G4).
`bin/observatory.js`	CLI ÇALIŞTIRICI (UNİX)	NPM global kurulumunda veya terminalde `observatory` komutu verildiğinde `src/cli.ts` dosyasını Node.js `--experimental-strip-types` argümanı ile başlatan ana giriş noktasıdır.
`bin/observatory.cmd`	CLI ÇALIŞTIRICI (WİN)	Windows ortamında PATH üzerinden `observatory` komutunun sorunsuz çalışmasını sağlayan CMD/PowerShell kabuk sarmalayıcısıdır.
`bin/pick_folder.exe`	YEREL YARDIMCI (WİN)	Web arayüzünde kullanıcının yerel bilgisayarındaki çalışma dizinini (CWD) görsel olarak seçebilmesi için Windows Forms `FolderBrowserDialog` açan hafif C# derlemesidir.
`README.md`	DOKÜMANTASYON	Projenin tanıtımı, hızlı başlangıç kılavuzu, mimari özellikleri, desteklenen 6 ajan motoru ve kullanım örneklerini içeren ana kılavuzdur.
`ARCHITECTURE.md`	DOKÜMANTASYON	Sistemin 3 katmanlı telemetri modelini, SQLite şemasını, SSE protokolünü ve bileşenler arası veri akışını detaylandıran temel mimari referans belgesidir.
`CHANGELOG.md`	SÜRÜM GEÇMİŞİ	Sürümler arasındaki değişiklikleri, yeni özellikleri (Antigravity ingest, tam ekran ofis, HUD paneli) ve hata düzeltmelerini Keep-a-Changelog standardında listeler.
`CLAUDE.md`	AJAN DİREKTİFLERİ	Claude Code ve diğer yapay zekâ asistanlarının projede çalışırken uyması gereken katı test, tip güvenliği ve kodlama standartlarını belirler.
`KNOWLEDGE.md`	BİLGİ TABANI	Sistemdeki özel çözümleri, Windows süreç yönetimi püf noktalarını, SQLite PRAGMA ayarlarını ve bilinen uç durumları saklar.
`ROADMAP.md`	YOL HARİTASI	Gelecek sürümlerde planlanan özellikleri, 3D ofis geliştirmelerini ve DAG orkestrasyon hedeflerini belgeler.
`TASKS.md`	GÖREV TAKİBİ	Tamamlanan ve devam eden geliştirme görevlerinin kontrol listesini içerir.
`SKILLS.md`	YETENEKLER	Observatory'nin ajanlara sunduğu araç ve yetenek kataloglarının açıklamasını barındırır.
`LICENSE`	LİSANS	Projenin açık kaynak MIT lisans metnini içerir.

3. Çekirdek Sistem Katmanı (src/core/)

Veri tutarlılığını, ACID kalıcılığı, durum geçişlerini ve görev matematiksel sıralamasını sağlayan kalptir.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`src/core/types.ts`	VERİ SÖZLEŞMELERİ	Sistem genelinde kullanılan tüm TypeScript veri tiplerini tanımlar: `EventType` (12 farklı olay türü), `AgentType` (claude, codex, agy, opencode vb.), `TaskEvent`, `TaskSummary`, `ToolCall`, `TaskUsage` ve `TaskStatus`. Bütün adaptörler bu sözleşmeye uyar.
`src/core/event_store.ts`	OLAY VERİ YOLU	Bellek içi (in-memory) olay dağıtımını yönetir. Dinleyicilere (listeners) canlı olay yayını yapar, olayları filtreler ve arka plandaki kalıcı depolama ile arayüzler arasında pub/sub köprüsü kurar.
`src/core/sqlite_store.ts`	KALICI DEPOLAMA	Node.js yerel `DatabaseSync` (SQLite) motorunu kullanarak `~/.observatory/observatory.sqlite` veritabanını yönetir. `tasks` ve `events` tablolarını oluşturur, WAL modunda atomik işlem yürütür. Önceden derlenmiş (prepared statement) önbelleği ve tek `BEGIN IMMEDIATE...COMMIT` işlemi altında toplu olay kaydı (`recordEvents`) ile sıfır kilitlenme ve disk I/O fırtınası önler; mükerrer olaylarda (dedup) gereksiz sorguları atlar.
`src/core/paths.ts`	YOL & BİNARY ÇÖZÜMLEME	İşletim sistemine duyarlı yol çözümleyicidir (Windows .cmd/.exe shimleri, PATH taraması). Ajan çalıştırılabilir dosyalarını (`claude`, `codex`, `opencode`, `agy`) ortam değişkenlerine göre hatasız tespit eder.
`src/core/state_machine.ts`	DURUM MAKİNESİ	Görevlerin durum geçişlerini deterministik kurallara bağlar (`pending → running → completed / failed / cancelled`). Terminal duruma geçmiş bir görevin bozulmasını engeller; kiralama (lease) süreleri ve çökme kurtarma (crash recovery) kontrollerini sağlar.
`src/core/dag_scheduler.ts`	DAG ÇİZELGELEYİCİ	Birden çok ajanın ardışık veya paralel çalışacağı görev ağlarını (Directed Acyclic Graph) yönetir. Graf döngülerini (circular dependency) tespit edip engeller, görevleri topolojik dalgalar (waves) halinde paralel yürütür.

Node.js yerel `DatabaseSync` (SQLite) motorunu kullanarak `~/.observatory/observatory.sqlite` veritabanını yönetir. `tasks` ve `events` tablolarını oluşturur, WAL modunda atomik işlem yürütür, tekilleştirme (dedup) ve kümülatif token/araç metriklerini günceller.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`src/core/failure.ts`	HATA SINIFLANDIRICISI	Ajan çıkış kodlarını ve hata mesajlarını sınıflandırır: `rate_limit` (429 kota aşımı), `auth` (kimlik hatası), `model_unsupported` (geçersiz model), `permission` (yetki reddi) ve `spawn_failed`.
`src/core/mission_contract.ts`	GÖREV SÖZLEŞMESİ	Çoklu ajan filolarına gönderilen görev sözleşmelerini (Mission Contract) doğrular. Bütçe, zaman aşımı sınırı, takım kısıtları ve doğrulama kapısı kriterlerini denetler.
`src/core/mission_types.ts`	GÖREV TİPLERİ	Görev sözleşmelerinin, takım tanımlarının (Kırmızı, Turuncu, Mavi, Siyah) ve ajan yetenek matrisinin (`code_write`, `code_review`, `terminal_exec`, `browser_automation`) tip tanımlarını barındırır.
`src/core/claude_session_manager.ts`	OTURUM YÖNETİCİSİ	Claude Code oturumlarının kalıcılığını ve devamlılığını (multi-turn resume) yönetir. Proje dizini bazında oturum kimliklerini eşler ve bağlam kaybı olmadan göreve devam edilmesini sağlar.

4. Ajan Adaptörleri Katmanı (src/adapters/)

Farklı yapay zekâ CLI araçlarının komut satırı sözdizimini ve çıktı formatlarını standart bir arayüze dönüştürür.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`src/adapters/base_adapter.ts`	SOYUT TABAN SINIF	Bütün adaptörlerin türediği temel sınıftır. `buildCommand(opts)`, `handleStdoutLine()`, `probe()`, `getCapabilities()` ve `supports()` fonksiyonlarının sözleşmesini belirler.
`src/adapters/claude_adapter.ts`	CLAUDE CODE ADAPTÖRÜ	Anthropic Claude Code CLI (`claude`) için komut argümanlarını (`--print`, `--dangerously-skip-permissions`, `--resume`, `--thinking-effort`) hazırlar. Akış çıktısındaki NDJSON karelerini (araç çağrıları, jeton kullanımı, mesaj parçaları) ayrıştırarak standart `TaskEvent` üretir.
`src/adapters/codex_adapter.ts`	CODEX CLI ADAPTÖRÜ	OpenAI Codex CLI (`codex exec`) için argümanları yapılandırır. Çıktıdaki JSON olaylarını (`item.started`, `aggregated_output`, dosya farkları ve komut onayları) takip ederek sisteme aktarır.
`src/adapters/opencode_adapter.ts`	OPENCODE ADAPTÖRÜ	OpenCode CLI (`opencode run`) için komut üretir. `opencode/deepseek-v4.1-flash-free` ve diğer ücretsiz modelleri destekler; araç adımlarını, girdi/çıktılarını ve adım bitişlerini yakalar.
`src/adapters/agy_adapter.ts`	ANTİGRAVİTY ADAPTÖRÜ	Google Antigravity CLI (`agy -p`) için çalıştırıcıdır. Gemini tabanlı otonom yazılım ajanlarının çalışma adımlarını ve araç çıktılarını standart olaylara dönüştürür.
`src/adapters/commandcode_adapter.ts`	COMMAND CODE ADAPTÖRÜ	Command Code aracı için CLI argümanlarını oluşturur ve terminal çıktılarını dinler.
`src/adapters/freebuff_adapter.ts`	FREEBUFF ADAPTÖRÜ	Freebuff açık kaynak CLI motoru için adaptasyon katmanıdır.

5. Yürütme ve Çalışma Zamanı Katmanı (src/runtime/)

Süreçlerin yaşam döngüsünü, paralel filo yönetimini, Git worktree izolasyonunu ve eşzamanlılık kilitlerini yönetir.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`src/runtime/agent_runner.ts`	SÜREÇ YÜRÜTÜCÜSÜ	Adaptörler tarafından hazırlanan komutları `child_process.spawn` ile çalıştırır. Canlı stdout/stderr akışını dinler, zaman aşımı ve iptal (cancellation) durumunda süreç ağacını güvenle sonlandırır, olay deposuna yazar.
`src/runtime/direct_runner.ts`	DOĞRUDAN GÖREV KOŞUCUSU	Gelen görev sözleşmesine uygun ajanı otomatik seçer (yetenek filtreleme), kota hatası alan ajanları dinlenme süresine (cooling off) çeker ve görev sonucunu doğrudan SQLite'a kaydeder.
`src/runtime/fleet_orchestrator.ts`	FİLO ORKESTRATÖRÜ	4 ayrı takımdan (Kırmızı Takım: Codex/Güvenlik, Turuncu Takım: Claude/Kodlama, Mavi Takım: Antigravity/Mimari, Siyah Takım: OpenCode/Veri) oluşan ajan filosunu DAG grafiğine göre koordine eder.
`src/runtime/resource_broker.ts`	KAYNAK ARACISI & KİLİT	Aynı kod tabanında çalışan paralel ajanların birbirinin dosyalarını ezmesini engeller. Eşzamanlılık semaforları, entegrasyon mutex'leri ve görev kiralama çitleri (fencing) sağlar.
`src/runtime/worktree_manager.ts`	GİT WORKTREE YÖNETİCİSİ	Her ajana ana projeden izole edilmiş geçici bir Git çalışma dalı (`git worktree add`) açar. Ajan işini bitirip testleri geçince değişiklikleri ana dala temizce birleştirir (merge gate) ve worktree'yi siler.
`src/runtime/tui_launcher.ts`	TUI BAŞLATICI	Terminal kullanıcı arayüzünü (TUI) bağımsız bir pencerede veya ayrık terminal oturumunda (pty detachment) başlatır.
`src/runtime/pick_folder.cs`	KLASÖR SEÇİCİ KAYNAK	Windows ortamında yerel klasör seçim penceresini tetikleyen C# kaynak kodudur. `bin/pick_folder.exe` ikili dosyasını üretir.
`src/runtime/pick_folder.ps1`	KLASÖR SEÇİCİ SCRİPT	C# ikili dosyasının bulunamadığı durumlarda PowerShell üzerinden yerel GUI klasör seçici penceresi açan yedek scripttir.

6. Harici Oturum İzleme Katmanı (src/ingest/)

Geliştiricinin terminalde elle bağımsız başlattığı veya Antigravity IDE içinden yürüttüğü oturumları otomatik içeri aktarır (Tier 3).

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU	
`src/ingest/core.ts`	İZLEME ÇEKİRDEĞİ	Dış kaynaklardan toplanan olaylar için `ExtEvent`, `LineParser` ve `ManagedCheck` arayüzlerini tanımlar. Metin düzleştirme, enjekte metin temizliği ve görev çiftleme kontrollerini sağlar.	
`src/ingest/antigravity.ts`	ANTİGRAVİTY İZLEYİCİSİ	Antigravity'nin `~/.gemini/antigravity/brain/*/transcript.jsonl` loglarını satır satır okur. `extractAntigravityPrompt` ile `<USER_REQUEST>` temizliğini yapar; `extractAntigravityModel` ile `<USER_SETTINGS_CHANGE>` etiketinden aktif modeli dinamik olarak tespit eder (örn. `gemini-3.8-flash`). `USER_INPUT`, `PLANNER_RESPONSE` (araçlar, tokenlar) ve `GENERIC` (çıktılar) olaylarını yakalayıp Observatory SQLite'a canlı akıtır.	Antigravity'nin `~/.gemini/antigravity/brain/*/transcript.jsonl` loglarını satır satır okur. `USER_INPUT`, `PLANNER_RESPONSE` (araçlar, tokenlar) ve `GENERIC` (çıktılar) olaylarını yakalayıp Observatory SQLite'a anlık akıtır.
`src/ingest/claude.ts`	CLAUDE CODE İZLEYİCİSİ	Kullanıcının `~/.claude/projects/` altındaki JSONL loglarını okur. Ana ajan adımlarını ve `subagents/` klasöründeki alt ajan hiyerarşilerini tespit eder.	
`src/ingest/codex.ts`	CODEX İZLEYİCİSİ	`~/.codex/sessions/` altındaki oturum kayıtlarını tarar. Terminalde elle çalıştırılan Codex komutlarını ve diff onaylarını SQLite'a taşır.	
`src/ingest/opencode.ts`	OPENCODE İZLEYİCİSİ	OpenCode'un yerel SQLite veritabanını (`~/.local/share/opencode/opencode.db`) yoklar (poll). `session`, `message` ve `part` tablolarından kullanıcı istemlerini ve araç çağrılarını çeker.	
`src/ingest/index.ts`	İZLEME KOORDİNATÖRÜ	Tüm izleyicileri (claude, codex, opencode, agy) koordine eden ana arka plan motorudur. `startIngest` ve `runIngest` fonksiyonlarını barındırır. Dosya başına toplu dizi (batch) oluşturarak tek transaction ile yazar, 60 dakikalık aktif pencere (`activeMinutes: 60`) ile açılış donmasını engeller ve `walkJsonl` klasör filtrelemesi ile event loop kilitlenmesini önler.	Tüm izleyicileri (claude, codex, opencode, agy) koordine eden ana arka plan motorudur. `startIngest` ve `runIngest` fonksiyonlarını barındırır; dosya ağaçlarını izler ve arka plan döngüsünü yönetir.

7. Komutlar ve Servisler Katmanı (src/commands/)

HTTP API'leri, gerçek zamanlı SSE veri akışını, sistem denetimini ve web servislerini sunar.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`src/commands/serve.ts`	HTTP & SSE SUNUCUSU	REST ve Server-Sent Events (SSE) sunucusunu başlatır. `/tasks`, `/events`, `/health` uçlarını sağlar. Web paneli açıldığında arka planda otomatik ingest motorunu devreye alır.
`src/commands/web.ts`	WEB ARAYÜZ SUNUCUSU	Web sohbet ve operasyon paneli sunucusudur. Statik HTML/CSS/JS dosyalarını sunar; `/api/run` (ajan çalıştırma), `/api/approve` (onay kapısı) ve model listeleme uçlarını yönetir.
`src/commands/doctor.ts`	SİSTEM DOKTORU	`observatory doctor [--live]` komutudur. Kurulu olan tüm CLI araçlarının varlığını, sürümlerini, kimlik doğrulama durumlarını ve canlı yanıt sürelerini denetler.
`src/commands/office_opener.ts`	OFİS AÇICI	`observatory office` komutunu çalıştırarak terminal tabanlı ASCII ofis görünümünü ekrana getirir.
`src/commands/web_opener.ts`	TARAYICI BAŞLATICI	Web sunucusu hazır olduğunda kullanıcının varsayılan tarayıcısını güvenlik jetonu (token) ile birlikte otomatik olarak açar.
`src/commands/web_remote.ts`	UZAKTAN ERİŞİM & MOBİL	Web arayüzünün yerel ağda (LAN) veya Tailscale üzerinden güvenli açılmasını sağlar. Tek kullanımlık eşleştirme kodları ve QR kod doğrulamasını yönetir.
`src/commands/web_capabilities.ts`	YETENEK TARAYICISI	Kullanıcının sisteminde kurulu olan becerileri (skills), MCP araçlarını, eklentileri ve modelleri tarayıp web arayüzüne JSON olarak aktarır.
`src/commands/web_sessions.ts`	OTURUM YÖNETİMİ	Web arayüzü için geçmiş oturumların listelenmesi, aranması ve ayrıntılarının çekilmesi işlemlerini yürütür.
`src/commands/web_snapshot.ts`	GİT ANLIK GÖRÜNTÜ	Riskli bir ajan görevi başlatılmadan önce çalışma dizininin geçici bir Git snapshot'ını alır; görev başarısız olursa tek tuşla geri alma (rollback) imkânı sunar.

8. Terminal UI, MCP Sunucusu ve Telemetri (src/ui/, src/mcp/, src/telemetry/)

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU	
`src/ui/tui.ts`	TERMİNAL ÇEKİRDEĞİ	ANSI renk biçimlendirmeleri (paint), metin kırpma, süre formatlama (hms), dönen yükleme simgeleri (spinners) ve terminal boyut hesaplayıcılarını içerir.	
`src/ui/tree.ts`	TUI AĞAÇ PANOSU	`observatory tree` komutunun ana görünümüdür. En güncel oturumlar en üstte başlar ve imleç ilk görevde konumlanır. Dikey kaydırma çubuğu (`scrollOffset`, ▏ iz ve ▉ tutamaç), fareyle tutup sürükleme (click & drag), çoklu mesaj oturum başlıklarında başlangıç saati damgası (`başladı HH:MM:SS`) ve 80ms akıcı Braille spinner animasyonu içerir. `[A]` filtreleme, `[Space]` katlama ve `[Enter]` ayrıntı sunar.	`observatory tree` komutunun ana görünümüdür. Çalışan ve son 24 saatte tamamlanan görevleri hiyerarşik ağaç şeklinde gösterir. `[A]` ile tüm geçmişi listeler, `[Enter]` ile ajan detayına iner.
`src/ui/transcript.ts`	DÖKÜM FORMATLAYICI	Bir göreve ait tüm kancaları, araç çağrılarını ve metin yanıtlarını terminalde renkli, okunaklı bir döküm (transcript) olarak çizer.	
`src/ui/skins.ts`	TAKIM & AJAN TEMALARI	Takım renk kodlarını (Kırmızı, Turuncu, Mavi, Siyah), emoji rozetlerini ve ajan simgelerini yönetir.	
`src/ui/office_view.ts`	ASCII OFİS GÖRÜNÜMÜ	Terminal içinde metin tabanlı (ASCII/ANSI) bir sanal ofis çizer; masalarda oturan ajanların çalışma ve bekleme durumlarını simüle eder.	
`src/mcp/mcp_server.ts`	RESMİ MCP SUNUCUSU	Antigravity ve diğer MCP istemcilerine `run_agent`, `start_task`, `get_task`, `cancel_task`, `list_tasks` ve `run_fleet` gibi araçları sunan Model Context Protocol sunucusudur.	
`src/telemetry/metrics.ts`	PERFORMANS METRİKLERİ	Ajanların yanıt sürelerini (p50, p90, p99 gecikme), başarı oranlarını, token harcamalarını hesaplar ve en uygun ajanı seçmek için yönlendirme skoru üretir.	
`src/cli.ts`	MERKEZİ CLI YÖNLENDİRİCİ	Komut satırından girilen tüm parametreleri (`tree`, `web`, `doctor`, `ingest`, `serve`, `mcp`) ayrıştırıp ilgili alt modüllere yönlendiren ana giriş noktasıdır.	
`src/index.ts`	KÜTÜPHANE GİRİŞİ	Agent Observatory'nin harici Node.js projelerinden bir kütüphane (SDK) olarak çağrılabilmesi için temel sınıfları ve fonksiyonları dışa aktarır (export).	

9. Web Kullanıcı Arayüzü ve Canlı HUD (web/)

Tarayıcıda çalışan, 2D piksel sanatlı ofisi, konuşma balonlarını, 4 sekmeli operasyon HUD'unu ve sohbet arabirimini sunar.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`web/index.html`	WEB İSKELETİ	Uygulamanın ana HTML yapısıdır. Sohbet alanı, yan menü, tam ekran ajan operasyon panosu, sanal ofis canvas'ı ve 4 sekmeli HUD kapsayıcısını içerir.
`web/app.css`	ANA STİL DOSYASI	Koyu/açık tema değişkenleri, responsive ızgara yerleşimi, tam ekran çekmece animasyonları ve HUD sekmeleri için şık CSS stillerini barındırır.
`web/app.js`	ANA İSTEMCİ KONTROLCÜSÜ	İstemci tarafının beynidir. SSE bağlantısını yönetir, akış olaylarını yakalar, HUD sekmelerini (`renderHudStream`, `renderHudDetail`, `renderHudFiles`, `renderHudTasks`) günceller ve tam ekran ofis modunu koordine eder.
`web/office_sim.js`	PİKSEL OFİS SİMÜLATÖRÜ	HTML5 Canvas üzerinde 2D piksel ofisi çizer. Ajanların masalarına yürümesini, kahve içmesini, kod yazmasını ve kafalarının üstünde pikselli konuşma balonları (`drawSpeechBubble`) çıkarmasını sağlar.
`web/dag_view.js`	DAG GRAF GÖRSELLEŞTİRİCİ	Paralel ve ardışık çoklu ajan görevlerini interaktif SVG düğüm grafiği olarak render eder.
`web/fleet_view.js`	FİLO PANOSU	4 takımın (Kırmızı, Turuncu, Mavi, Siyah) mevcut durumunu, çalışan ajanlarını ve canlı araç çağrılarını kartlar halinde gösterir.
`web/diff.js`	KOD FARKI MOTORU	Ajanların oluşturduğu veya değiştirdiği kodların unified git diff karşılaştırmasını ve satır numaralandırmasını hesaplar.
`web/diff_viewer.js`	FARK İNCELEME MODALI	Kod değişikliklerini sekmeli dosya gezgini halinde sunan, kullanıcının tek tıkla onaylamasını veya reddetmesini sağlayan modal arayüzdür.
`web/mission_composer.js`	GÖREV BESTECİSİ	Kullanıcının görsel bir form üzerinden çoklu ajan görev sözleşmesi (Mission Contract) oluşturup göndermesini sağlayan arayüzdür.
`web/subagents.js`	ALT AJAN AĞACI	Ana ajanın başlattığı alt ajanların (subagents) hiyerarşisini, sürelerini ve araç kullanımını katlanabilir ağaç yapısında çizer.
`web/context.js`	BAĞLAM YÖNETİCİSİ	Sohbet bağlamını, sistem talimatlarını, jeton bütçesini ve yeniden deneme kurallarını denetler.
`web/runcard.js`	ÇALIŞMA KARTI	Yürütülen görevin adımlarını, canlı süresini, kullanılan araçların ayrıntılarını gösteren dinamik durum kartıdır.
`web/sessions.js`	OTURUM YÖNETİCİSİ	Geçmiş sohbetlerin sol kenar çubuğunda listelenmesini, aranmasını, yeniden adlandırılmasını ve silinmesini yönetir.
`web/projects.js`	PROJE YÖNETİCİSİ	Farklı çalışma dizinleri ve yerel projeler arasında geçiş yapılmasını ve son projelerin hatırlanmasını sağlar.
`web/router.js`	AJAN YÖNLENDİRME	İstemin içeriğine göre en uygun ajanı otomatik seçen güvenlik ve yönlendirme kurallarını içerir.
`web/skins.ts / skins.js`	GÖRSEL TEMALAR	Ajan avatarlarını, takım renklerini ve araç etiketlerinin simgelerini eşleyen görsel kütüphanedir.
`web/store.js`	YEREL DEPOLAMA (STORAGE)	Tarayıcının `localStorage` belleğini kullanarak sohbet geçmişini ve kullanıcı tercihlerini saklar.

DOSYA YOLU	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`web/md.js`	GÜVENLİ MARKDOWN	Ajanlardan gelen markdown yanıtlarını güvenle HTML'e dönüştürür; zararlı XSS betiklerini ve tehlikeli bağlantıları engeller.
`web/cup.js`	KAHVE FİNCANI ANİMASYONU	Boşta (idle) duran ajanların masasında tüten piksel kahve fincanı animasyonunu yönetir.
`web/i18n.js, lang-*.js`	ÇOKLU DİL MOTORU	Türkçe (`lang-tr.js`) ve İngilizce (`lang-en.js`) dil paketleri arasında anlık geçiş yapılmasını sağlayan reaktif çeviri motorudur.
`web/qr.js, pair.html/.js`	MOBİL EŞLEŞTİRME	Harici kütüphane kullanmadan çevrimdışı SVG QR kod üreterek telefonun web arayüzüne anında bağlanmasını sağlar.
`web/theme.js`	TEMA SEÇİCİ	Kullanıcının karanlık veya aydınlık arayüz temasını seçmesini ve tarayıcıda kalıcı olmasını sağlar.

10. Ek Modüller, Dokümantasyon ve Test Mimarisi

DİZİN / DOSYA	KATEGORİ	TEKNİK GÖREVİ VE SORUMLULUĞU
`docs/wiki/ (01-14.md)`	MİMARİ DOKÜMANTASYON	Agent Observatory'nin çalışma mekanizmasını, kurulumunu, MCP araçlarını, adaptörlerini, veritabanı şemasını, SSE API'sini ve mobil erişimini anlatan 14 bölümlük resmi kılavuz dizisidir.
`nexus_ai_office/`	DENEYSEL 3D OFİS	Babylon.js / Three.js tabanlı, uzay istasyonu temalı deneysel 3D sanal ofis prototipidir (Core Engine, Features_3D, TechBase_SpaceStation_3D).
`nexus_ai_office/tui_office/`	PYTHON TUI OFİS	Python tabanlı, bağımsız terminal simülasyonudur. ASCII ofis haritası (`office_map.py`), karakter çizimi (`renderer.py`) ve işçi döngülerini (`worker.py`) içerir.
`updates-dashboard/`	GÜNCELLEME PANELİ	Sistem sürüm notlarını ve değişiklik günlüklerini görsel olarak sunan hafif bağımsız mikro gösterge panelidir.
`test/ (41 Test Modülü)`	DOĞRULAMA SÜİTİ	Sistemin tüm katmanlarını izole in-memory SQLite ve sahte süreçlerle denetleyen kapsamlı test dizinidir: • `verify_core.ts`: Güvenlik, dizin denetimi, paralel oturum izolasyonu ve iptal garantileri. • `tui_lag_benchmark.test.ts`: TUI ilk açılış süresi (cold render < 100ms), ingest tick ve `monitorEventLoopDelay` gecikme/takılma profil doğrulaması. • `ingest.test.ts` & `ingest_subagents.test.ts`: Claude, Codex, OpenCode ve Antigravity model/istem ayrıştırma doğrulaması. • `serve.test.ts` & `web.test.ts`: HTTP uçları, Origin/Host korumaları ve SSE akış bütünlüğü. • `tree_session.test.ts` & `office_view.test.ts`: TUI kaydırma çubuğu, başlangıç saati ve ofis çizim mantığı. • `dag_scheduler.test.ts` & `worktree_acceptance.test.ts`: Paralel filo dalgaları ve Git merge doğrulaması.

> **Sonuç ve Doğrulama Durumu**
> Agent Observatory kod tabanındaki tüm dosyalar mimari prensiplere uygun olarak incelenmiş, aralarındaki veri bağımlılıkları teyit edilmiş ve 41 test modülünden oluşan tam süitin %100 başarıyla geçtiği doğrulanmıştır.

Lifecycle and TUI Performance Update

29 October 2026 - authoritative addendum

src/core/sqlite_store.ts

reapStaleTasks(maxAgeMs = 300000) examines running tasks using the latest event timestamp, falling back to start time. Five minutes of inactivity or a confirmed missing PID (ESRCH) produces task.cancelled with the session-ended / timeout reason. Permission errors do not prove process death. Selection, event insertion and summary updates share BEGIN IMMEDIATE / COMMIT with rollback. No process is killed. getTaskRevision combines data_version and total_changes to detect local and external writes.

src/ingest/index.ts

The shared ingest tick reads new source events before running reconciliation at most once every 30 seconds. This covers startBackgroundIngest, CLI follow, web and tree consumers. A producer crash without a known PID is detected through inactivity, not immediate process observation.

src/ingest/opencode.ts

User message IDs group multiple text parts into one turn. A new user turn cancels an unfinished predecessor. Recorded assistant errors fail the open turn. Normal step-finish stop completes it. Closing this read-only observer does not imply that OpenCode has exited; missing stop events use store reconciliation.

src/ui/tree.ts

Startup reconciles stale tasks. Animation and spinner cadence is 200 ms. Animation uses cached task and detail data without SQLite reads; the 800 ms refresh checks database revisions before loading summaries and invalidating detail data. diffTreeLines sends only changed rows and clears removed rows. Resize forces a complete redraw.

Regression coverage and change record

test/ingest.test.ts covers stale activity, recent events, terminal-state preservation, repeated cleanup, PID error semantics and multi-part OpenCode turns. test/tree_session.test.ts covers changed, identical, added and removed rows. CHANGELOG.md records the lifecycle and rendering changes. Tests use synthetic fixtures and in-memory stores for the new lifecycle cases.

Validation scope

This addendum supersedes any earlier blanket production-readiness claim for these changes. TypeScript and automated test results are reported separately in the delivery. Interactive terminal behavior and real producer crashes are not certified by unit tests. The five-minute policy also cancels a live process if it emits no events for that period.

Performance Pass: TUI, Ingest, SQLite (2026-10-29)

Mimari ve Kod De■i■ikli■i Güncellemesi

src/ui/tree.ts: new **TranscriptView** (incremental transcript: events rendered once, finished lines wrapped once per width), **loadDetail / refreshDetail / DetailState** (detail page loads the newest DETAIL_EVENT_LIMIT=5000 events and appends only rows after the last seq on DB revision change). clipAnsi/wrapAnsi use a sticky regex instead of O(n^2) slicing; cleaned assistant text is memoized per task object.

src/ingest/index.ts: periodic discovery runs asynchronously via fs.promises in live mode (option **asyncDiscover**, first scan stays synchronous); live reads are capped at 2 MB per file per tick and cut at the last newline. **src/ingest/opencode.ts**: poll is skipped while PRAGMA data_version is unchanged; statements are prepared once.

src/core/sqlite_store.ts: statement cache (prep), new indexes idx_events_session and idx_events_task_ts, tools_json appended without parse on tool.invoked, parsed tool lists cached per task. New methods: **getEventsTail**, **countEvents**, **getSessionTaskIndex**.

src/ui/tui.ts: seenEventIds bounded to 20000. **test/perf_paths.test.ts**: regression test (incremental view equals full render, tools_json append, tail queries).

Web UI Thinking Accordion and Dynamic Topic Selection

Mimari ve Kod De■i■ikli■i Güncellemesi

This update separates internal reasoning/thinking (CoT) from assistant text into collapsible details-summary accordion widgets in the web chat UI. It also replaces the blind Turn 1 session title lock with a multi-factor prompt scoring algorithm (pickSessionTitle) matching ChatGPT, Claude and Cursor standards, properly filtering greetings and noise.